

Trabalho Prático 1 - Algoritmo do Banqueiro

Curso: Sistemas Operacionais

Alunos: Caio Félix e Milena Cardoso

Projeto: Implementação do Algoritmo do Banqueiro com execução concorrente em Java

Data: 18/04/2026

Repositório

<https://github.com/milenacrd/trabalho-SO.git>

1. Introdução

O Algoritmo do Banqueiro é uma técnica clássica de prevenção de deadlocks em sistemas operacionais. A ideia central é permitir uma alocação de recursos apenas quando o sistema permanece em estado seguro, isto é, quando existe pelo menos uma sequência de finalização para todos os processos/clientes.

Neste trabalho, o algoritmo foi implementado para um cenário com 5 clientes e 3 tipos de recursos, com suporte a solicitações e liberações de recursos em ambiente concorrente. A relevância prática está em evitar condições de bloqueio permanente, mantendo a consistência do estado global do sistema mesmo com múltiplas threads executando ao mesmo tempo.

2. Desenvolvimento

2.1 Estruturas de dados

A implementação foi feita em Java no arquivo `src/BankersAlgorithm.java` com as seguintes estruturas principais:

- `available[3]`: quantidade de recursos disponíveis por tipo.
- `maximum[5][3]`: demanda máxima declarada por cliente.
- `allocation[5][3]`: recursos atualmente alocados por cliente.
- `need[5][3]`: recursos ainda necessários por cliente.

Relação fundamental usada no código:

$need = maximum - allocation$

2.2 Funções implementadas

As funções principais do sistema são:

- `initData(args)`: lê recursos da linha de comando, valida entrada e inicializa matrizes.
- `isSafe()`: executa o algoritmo de segurança simulando finalização de clientes.
- `requestResources(customerNum, request)`: valida limites, simula alocação, chama `isSafe()` e faz rollback se o estado ficar inseguro.
- `releaseResources(customerNum, release)`: valida liberação e atualiza `available`, `allocation` e `need`.
- `runSimulation()`: inicia 5 threads de clientes, cada uma realizando iterações de request/release.

2.3 Uso de mutex (sincronização)

A sincronização é feita com `ReentrantLock` global:

- `lock.lock()` e `lock.unlock()` protegem o acesso aos vetores e matrizes compartilhados.
- As operações request, release e verificação de segurança são executadas em seção crítica.
- Essa abordagem evita condições de corrida durante leituras/escritas concorrentes em `available`, `allocation` e `need`.

3. Resultados

3.1 Exemplos de execução (prints)

Exemplo A - Request negado por exceder necessidade:

```
```text
```

Comando:

```
java -cp src BankersAlgorithm 5 5 5 request 0 2 2 2
```

Saida relevante:

Cliente 0 - Request [2, 2, 2] negado: excede necessidade [2, 1, 2]

Resultado: Negado

```
```
```

Exemplo B - Request negado por estado inseguro:

```
```text
```

Comando:

```
java -cp src BankersAlgorithm 1 1 1 request 0 1 0 0
```

Saida relevante:

Cliente 0 - Request [1, 0, 0] negado: causaria estado inseguro.

Resultado: Negado

...

Exemplo C - Simulação concorrente:

```text

Comando:

java -cp src BankersAlgorithm 10 5 7 simulate

Saida relevante:

[Cliente X] request negado: ...

...

Simulacao finalizada.

Estado seguro? false

```

### 3.2 Testes realizados

Com base em tests/RelatorioTestesFinal.txt e tests/README.md, a fase de testes validou:

- Mini Task 3.2: algoritmo de segurança (5 testes).
- Mini Task 3.3: request/release (4 testes).
- Mini Task 3.4: cenários de deadlock (4 testes).
- Mini Task 3.5: validação multithread (11+ execuções).
- Mini Task 3.6: carga e performance (12 rodadas).
- Mini Task 3.7: documentação e cobertura.
- Mini Task 3.8: bateria completa final.

Resultado consolidado:

- Testes automatizados da suite Java: 6/6 aprovados.
- Taxa de sucesso global: 100%.
- Falhas: 0.
- Carga/performance estavel em PAR=1,2,4,8 com:
  - LEVEL=1 AVG\_REQ\_S=15.16
  - LEVEL=2 AVG\_REQ\_S=30.78
  - LEVEL=4 AVG\_REQ\_S=46.74
  - LEVEL=8 AVG\_REQ\_S=67.83
- Em todas as rodadas da bateria final:
  - ALL\_SAFE=True
  - ALL\_FINALIZE=True
  - ANY\_EXCEPTION=False

## 4. Conclusão

O trabalho atingiu o objetivo de implementar e validar o Algoritmo do Banqueiro em ambiente concorrente, com foco em prevenção de deadlocks e consistência de estado.

Lições aprendidas:

- A sincronização é essencial em sistemas multithread para evitar race conditions.
- O algoritmo de segurança funciona como barreira preventiva, mesmo quando isso reduz a taxa de aprovações de requests.
- Testes de carga e repetibilidade são importantes para validar estabilidade além da correção funcional.

Dificuldades encontradas:

- Tratamento de encoding em arquivos de teste (UTF-8 sem BOM).
- Ajustes de package/classpath para execução correta da suite Java.
- Limitação do estado inicial (allocation zerado), exigindo estratégias de teste complementares para alguns cenários de release.

Status final do projeto: APROVADO para entrega no escopo definido.

## **5. Aviso sobre uso de IA**

Parte do código e da documentação foi gerada com apoio de IA (LLM), com revisão humana e validação por testes automatizados e manuais antes da entrega.